

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №3
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Динамические списки»

Выполнил:

студенты группы 24ВВВ2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

доцент Юрова О.В.

к.н. Деев М.В.

Пенза 2025

Название

Динамические списки.

Цель работы

Научиться правильно реализовывать весь функционал динамических списков.

Лабораторное задание

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. На основе приведенного кода реализуйте структуру данных Очередь.
3. На основе приведенного кода реализуйте структуру данных Стек.

Описание метода решения задачи

Метод решения основан на реализации динамических связанных списков с использованием указателей и malloc/free. Для всех трех структур используется общий подход: создается структура node с полями данных и указателем next. Ключевое различие - в логике операций добавления и удаления элементов.

Приоритетная очередь использует упорядоченную вставку по приоритету - новый элемент сравнивается с существующими и размещается в соответствующей позиции. Очередь FIFO работает по принципу "первый пришел - первый вышел" с двумя указателями на начало и конец. Стек LIFO использует один указатель на вершину и работает по принципу "последний пришел - первый вышел".

Все программы имеют единую архитектуру: интерактивное меню с защитой ввода, поддержка русского языка, проверка ошибок памяти и ввода, обязательная очистка памяти перед выходом.

Листинг

Файл 3LR_1P.cpp (1 задание)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <windows.h>
#include <wchar.h>
#include <ctype.h>

//Структура элемента приоритетной очереди
struct node {
    wchar_t inf[256]; // Название объекта
    int priority; // Приоритет элемента
    struct node* next; // Указатель на следующий элемент
};

struct node* head = NULL; // Указатель на голову очереди
```

```

//Функция очистки буфера ввода
void clear_input_buffer() {
    int c;
    //Читаем символы из буфера до конца строки
    while ((c = getchar()) != '\n' && c != EOF);
}

//Функция проверки, является ли строка числом
int is_number(const char* str) {
    if (str == NULL || *str == '\0') {
        return 0; // Пустая строка - не число
    }
    char* endptr;
    //Пытаемся преобразовать строку в число
    strtol(str, &endptr, 10);
    //Если endptr указывает на конец строки - это число
    return *endptr == '\0';
}

//Функция проверки существования приоритета
int priority_exists(int priority) {
    struct node* current = head;
    while (current != NULL) {
        if (current->priority == priority) {
            return 1; // Приоритет уже существует
        }
        current = current->next;
    }
    return 0; // Приоритет не найден
}

//Функция создания нового элемента очереди
struct node* get_struct(void) {
    //Выделяем память под новый элемент
    struct node* p = (struct node*)malloc(sizeof(struct node));
    if (p == NULL) {
        wprintf(L"Ошибка при распределении памяти\n");
        exit(1);
    }

    //Ввод названия объекта
    wprintf(L"Введите название объекта: ");
    clear_input_buffer(); // Очищаем буфер перед чтением строки
    if (fgetws(p->inf, 256, stdin) == NULL) {
        wprintf(L"Ошибка ввода\n");
        free(p);
        return NULL;
    }

    //Удаление символа новой строки из введенной строки
    size_t len = wcslen(p->inf);
    if (len > 0 && p->inf[len - 1] == L'\n') {
        p->inf[len - 1] = L'\0';
    }
}

```

```

//Ввод приоритета с проверкой корректности
char input[256];
int valid_input = 0;
int priority = 0;
while (!valid_input) {
    wprintf(L"Введите приоритет: ");
    if (scanf("%255s", input) != 1) {
        wprintf(L"Ошибка ввода.\n");
        clear_input_buffer();
        continue;
    }

    //Проверка, что введено число
    if (!is_number(input)) {
        wprintf(L"Ошибка: введите целое число\n");
        clear_input_buffer();
        continue;
    }

    priority = atoi(input); // Преобразуем строку в число
    //Проверка, что число положительное
    if (priority <= 0) {
        wprintf(L"Ошибка: приоритет должен быть положительным
числом.\n");
        clear_input_buffer();
        continue;
    }

    //Проверка, что приоритет не существует
    if (priority_exists(priority)) {
        wprintf(L"Ошибка: приоритет %d уже существует. Введите
уникальный приоритет.\n", priority);
        clear_input_buffer();
        continue;
    }
    valid_input = 1; // Ввод корректен, выходим из цикла
}
p->priority = priority; // Устанавливаем приоритет
p->next = NULL; // Следующий элемент пока отсутствует
return p;
}

//Добавление элемента в соответствии с приоритетом
void spstore(void) {
    struct node* p = get_struct();
    if (p == NULL) return;
    if (head == NULL) {
        //Если список пуст - новый элемент становится головой
        head = p;
        return;
    }
    if (p->priority > head->priority) {
        //Если новый элемент имеет высший приоритет чем голова,
вставляем его в начало списка
        p->next = head;
        head = p;
        return;
    }
}

```

```

    }

    //Ищем место для вставки в соответствии с приоритетом
    struct node* current = head;
    while (current->next != NULL && current->next->priority >= p-
>priority) {
        current = current->next;
    }

    //Вставляем элемент на найденную позицию
    p->next = current->next;
    current->next = p;
}

//Просмотр содержимого приоритетной очереди
void review(void) {
    struct node* current = head;
    if (current == NULL) {
        wprintf(L"Список пуст\n");
        return;
    }
    wprintf(L"Содержимое приоритетной очереди:\n");
    //Проходим по всем элементам списка и выводим их
    while (current != NULL) {
        wprintf(L"Имя: %ls, Приоритет: %d\n", current->inf, current-
>priority);
        current = current->next;
    }
}

//Удаление элемента с наивысшим приоритетом (из начала очереди)
void delete_highest_priority(void) {
    if (head == NULL) {
        wprintf(L"Очередь пуста\n");
        return;
    }
    //Сохраняем указатель на удаляемый элемент
    struct node* temp = head;
    //Перемещаем голову на следующий элемент
    head = head->next;
    wprintf(L"Удален элемент: %ls (приоритет: %d)\n", temp->inf,
temp->priority);
    free(temp); // Освобождаем память удаляемого элемента
}

int main() {
    setlocale(LC_ALL, "ru_RU.UTF-8");
    SetConsoleOutputCP(CP_UTF8); // Кодировка вывода
    SetConsoleCP(CP_UTF8); // Кодировка ввода
    int choice;
    char input[256];
    wprintf(L"Реализация приоритетной очереди\n");

    do {
        wprintf(L"\nМеню:\n");
        wprintf(L"1. Добавить элемент\n");
    }

```

```

wprintf(L"2. Просмотреть очередь\n");
wprintf(L"3. Удалить элемент с высшим приоритетом\n");
wprintf(L"4. Выход\n");
wprintf(L"Выберите действие: ");

//Чтение выбора пользователя
if (scanf("%255s", input) != 1) {
    clear_input_buffer();
    wprintf(L"Неверный ввод. Попробуйте снова.\n");
    continue;
}

//Проверка, что введено число
if (!is_number(input)) {
    wprintf(L"Ошибка: введите число от 1 до 4. Попробуйте
снова.\n");
    clear_input_buffer();
    continue;
}

choice = atoi(input); // Преобразуем ввод в число
switch (choice) {
case 1:
    spstore();
    break;
case 2:
    review();
    break;
case 3:
    delete_highest_priority();
    break;
case 4:
    wprintf(L"Выход...\n");
    break;
default:
    wprintf(L"Неверный выбор. Введите число от 1 до 4.\n");
}
} while (choice != 4); // Продолжаем пока не выбран выход
//Очистка памяти - удаление всех элементов очереди
while (head != NULL) {
    delete_highest_priority();
}
return 0;
}

```

Файл 3LR_2P.cpp (2 задание)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <windows.h>
#include <wchar.h>
#include <ctype.h>
#include <string.h>

```

```

//Структура элемента очереди
struct node {
    wchar_t inf[256]; // Название объекта
    struct node* next; // Указатель на следующий элемент в очереди
};
struct node* front = NULL; // Указатель на начало очереди
struct node* rear = NULL; // Указатель на конец очереди

//Функция очистки буфера ввода от лишних символов
void clear_input_buffer() {
    int c;
    //Читаем все символы до конца строки
    while ((c = getchar()) != '\n' && c != EOF);
}

//Функция проверки, является ли строка корректным целым числом
int is_number(const char* str) {
    if (str == NULL || *str == '\0') {
        return 0; // Пустая строка - не число
    }
    char* endptr;
    //Преобразовываем строку в число
    strtol(str, &endptr, 10);
    //Если преобразование прошло до конца строки - это корректное
    число
    return *endptr == '\0';
}

//Функция создания нового элемента очереди
struct node* get_struct(void) {
    //Выделяем память под новый элемент
    struct node* p = (struct node*)malloc(sizeof(struct node));
    if (p == NULL) {
        wprintf(L"Ошибка при распределении памяти\n");
        exit(1);
    }

    //Ввод названия объекта с проверкой уникальности
    int valid_input = 0;
    while (!valid_input) {
        wprintf(L"Введите название объекта: ");
        clear_input_buffer(); // Очищаем буфер перед чтением строки
        if (fgetws(p->inf, 256, stdin) == NULL) {
            wprintf(L"Ошибка ввода\n");
            free(p);
            return NULL;
        }
        //Удаление символа новой строки из введенной строки
        size_t len = wcslen(p->inf);
        if (len > 0 && p->inf[len - 1] == L'\n') {
            p->inf[len - 1] = L'\0';
        }
        valid_input = 1; // Ввод корректен, выходим из цикла
    }

    p->next = NULL; // Новый элемент пока не ссылается на следующий

```

```

        return p;
    }

//Добавление элемента в конец очереди
void enqueue(void) {
    struct node* p = get_struct();
    if (p == NULL) return;
    if (rear == NULL) {
        //Если очередь пуста - новый элемент становится и началом и
концом
        front = rear = p;
    }
    else {
        //Добавляем элемент в конец очереди
        rear->next = p;
        rear = p; // Обновляем указатель на конец очереди
    }
    wprintf(L"Элемент добавлен в очередь: %ls\n", p->inf);
}

//Удаление элемента из начала очереди
void dequeue(void) {
    if (front == NULL) {
        wprintf(L"Очередь пуста\n");
        return;
    }

    //Сохраняем указатель на удаляемый элемент
    struct node* temp = front;
    //Перемещаем начало очереди на следующий элемент
    front = front->next;
    //Если после удаления очередь стала пустой
    if (front == NULL) {
        rear = NULL; // Обнуляем указатель на конец
    }
    wprintf(L"Удален элемент: %ls\n", temp->inf);
    free(temp); // Освобождаем память удаленного элемента
}

//Просмотр первого элемента очереди без удаления
void peek(void) {
    if (front == NULL) {
        wprintf(L"Очередь пуста\n");
        return;
    }
    wprintf(L"Первый элемент: %ls\n", front->inf);
}

//Просмотр всей очереди
void display(void) {
    struct node* current = front;
    if (current == NULL) {
        wprintf(L"Очередь пуста\n");
        return;
    }
    wprintf(L"Содержимое очереди:\n");

```



```

//Пройдем по всем элементам очереди и выводим их
while (current != NULL) {
    wprintf(L"%ls\n", current->inf);
    current = current->next;
}
}

int main() {
    setlocale(LC_ALL, "ru_RU.UTF-8");
    SetConsoleOutputCP(CP_UTF8); // Для вывода
    SetConsoleCP(CP_UTF8); // Для ввода
    int choice;
    char input[256];
    wprintf(L"Реализация очереди (FIFO)\n");
    do {
        wprintf(L"\nМеню:\n");
        wprintf(L"1. Добавить элемент\n");
        wprintf(L"2. Удалить элемент\n");
        wprintf(L"3. Просмотреть первый элемент\n");
        wprintf(L"4. Просмотреть всю очередь\n");
        wprintf(L"5. Выход\n");
        wprintf(L"Выберите действие: ");
        //Чтение выбора пользователя
        if (scanf("%255s", input) != 1) {
            clear_input_buffer();
            wprintf(L"Неверный ввод. Попробуйте снова.\n");
            continue;
        }
        //Проверка, что введено число
        if (!is_number(input)) {
            wprintf(L"Ошибка: введите число от 1 до 5. Попробуйте снова.\n");
            clear_input_buffer();
            continue;
        }
        choice = atoi(input); // Преобразуем строку в число

        switch (choice) {
            case 1:
                enqueue();
                break;
            case 2:
                dequeue();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                wprintf(L"Выход...\n");
                break;
            default:
                wprintf(L"Неверный выбор. Введите число от 1 до 5.\n");
        }
    }
}

```

```

    } while (choice != 5); // Продолжаем пока не выбран выход

    //Очистка памяти - удаление всех элементов очереди
    while (front != NULL) {
        dequeue();
    }
    return 0;
}

```

Файл 3LR_3P.cpp (3 задание)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <windows.h>
#include <wchar.h>
#include <string.h>

//Структура элемента стека
struct node {
    wchar_t inf[256]; // Название объекта
    struct node* next; // Указатель на следующий элемент в стеке
};
struct node* top = NULL; // Указатель на вершину стека

//Функция очистки буфера ввода от лишних символов
void clear_input_buffer() {
    int c;
    //Читаем все символы до конца строки
    while ((c = getchar()) != '\n' && c != EOF);
}

//Функция проверки, является ли строка корректным целым числом
int is_number(const char* str) {
    if (str == NULL || *str == '\0') {
        return 0; // Пустая строка - не число
    }
    char* endptr;
    //Пытаемся преобразовать строку в число
    strtol(str, &endptr, 10);
    //Если преобразование прошло до конца строки - это корректное
    число
    return *endptr == '\0';
}

//Функция создания нового элемента стека
struct node* get_struct(void) {
    //Выделяем память под новый элемент
    struct node* p = (struct node*)malloc(sizeof(struct node));
    if (p == NULL) {
        wprintf(L"Ошибка при распределении памяти\n");
        exit(1);
    }

    //Ввод названия объекта
    int valid_input = 0;

```

```

while (!valid_input) {
    wprintf(L"Введите название объекта: ");
    clear_input_buffer(); // Очищаем буфер перед чтением строки
    if (fgetws(p->inf, 256, stdin) == NULL) {
        wprintf(L"Ошибка ввода\n");
        free(p);
        return NULL;
    }

    //Удаление символа новой строки из введенной строки
    size_t len = wcslen(p->inf);
    if (len > 0 && p->inf[len - 1] == L'\n') {
        p->inf[len - 1] = L'\0';
    }
    valid_input = 1; // Ввод корректен, выходим из цикла
}

p->next = NULL; // Новый элемент пока не ссылается на следующий
return p;
}

//Добавление элемента на вершину стека
void push(void) {
    struct node* p = get_struct();
    if (p == NULL) return;
    //Новый элемент становится новой вершиной стека
    p->next = top; // Связываем новый элемент с предыдущей вершиной
    top = p; // Обновляем указатель на вершину стека
    wprintf(L"Элемент добавлен в стек: %ls\n", p->inf);
}

//Удаление элемента с вершины стека
void pop(void) {
    if (top == NULL) {
        wprintf(L"Стек пуст\n");
        return;
    }
    //Сохраняем указатель на удаляемый элемент (вершину стека)
    struct node* temp = top;
    //Перемещаем вершину стека на следующий элемент
    top = top->next;
    wprintf(L"Удален элемент: %ls\n", temp->inf);
    free(temp); // Освобождаем память удаленного элемента
}

//Просмотр верхнего элемента стека без удаления
void peek_stack(void) {
    if (top == NULL) {
        wprintf(L"Стек пуст\n");
        return;
    }
    wprintf(L"Верхний элемент: %ls\n", top->inf);
}

//Просмотр всего стека (от вершины к основанию)
void display_stack(void) {

```

```

struct node* current = top;
if (current == NULL) {
    wprintf(L"Стек пуст\n");
    return;
}
wprintf(L"Содержимое стека (сверху вниз):\n");
//Проходим по всем элементам стека и выводим их
while (current != NULL) {
    wprintf(L"%ls\n", current->inf);
    current = current->next;
}
}

//Проверка пустоты стека
int is_empty(void) {
    return top == NULL; // Возвращает 1 если стек пуст, 0 если нет
}

int main() {
    setlocale(LC_ALL, "ru_RU.UTF-8");
    SetConsoleOutputCP(CP_UTF8); // Для вывода
    SetConsoleCP(CP_UTF8); // Для ввода
    int choice;
    char input[256];
    wprintf(L"Реализация стека (LIFO)\n");

    do {
        wprintf(L"\nМеню:\n");
        wprintf(L"1. Добавить элемент (push)\n");
        wprintf(L"2. Удалить элемент (pop)\n");
        wprintf(L"3. Просмотреть верхний элемент (peek)\n");
        wprintf(L"4. Просмотреть весь стек\n");
        wprintf(L"5. Проверить пустоту стека\n");
        wprintf(L"6. Выход\n");
        wprintf(L"Выберите действие: ");

        //Чтение выбора пользователя
        if (scanf("%255s", input) != 1) {
            clear_input_buffer();
            wprintf(L"Неверный ввод. Попробуйте снова.\n");
            continue;
        }

        //Проверка, что введено число
        if (!is_number(input)) {
            wprintf(L"Ошибка: введите число от 1 до 6. Попробуйте снова.\n");
            clear_input_buffer();
            continue;
        }
        choice = atoi(input); // Преобразуем строку в число
        switch (choice) {
            case 1:
                push();
                break;
            case 2:

```

```

        pop();
        break;
    case 3:
        peek_stack();
        break;
    case 4:
        display_stack();
        break;
    case 5:
        if (is_empty()) {
            wprintf(L"Стек пуст\n");
        }
        else {
            wprintf(L"Стек не пуст\n");
        }
        break;
    case 6:
        wprintf(L"Выход...\n");
        break;
    default:
        wprintf(L"Неверный выбор. Введите число от 1 до 6.\n");
    }
} while (choice != 6); // Продолжаем пока не выбран выход
//Очистка памяти - удаление всех элементов стека
while (top != NULL) {
    pop();
}
return 0;
}

```

Текст к программе

1. 3LR_1P.cpp (1 задание)

Приоритетная очередь - это динамическая структура данных, где каждый элемент имеет числовой приоритет. Элементы автоматически упорядочиваются по убыванию приоритета при добавлении, что гарантирует что элемент с наивысшим приоритетом всегда находится в начале очереди. Удаление происходит только из начала, что обеспечивает быстрый доступ к наиболее важным элементам. Реализация использует связанный список с упорядоченной вставкой, где новый элемент сравнивается с существующими и размещается в позицию, соответствующей его приоритету. Это делает структуру идеальной для задач планирования, где разные задачи имеют разный уровень важности.

2. 3LR_2P.cpp (2 задание)

Очередь FIFO - это классическая очередь работающая по принципу “первый пришел - первый вышел”. Элементы добавляются в конец и удаляются из начала структуры, что обеспечивает строгое соблюдение порядка поступления. Реализация использует два указателя: один на начало очереди, другой на конец, что позволяет эффективно выполнять операции добавления и удаления без необходимости полного перебора элементов. Такая организация делает очередь идеальной для обработки запросов в порядке их поступления, буферизации данных и любых сценариев, где важен порядок поступления.

элементов.

3. 3LR_3P.cpp (3 задание)

Стек LIFO - это структура данных работающая по принципу “последний пришел - первый вышел”. Все операции выполняются только с вершиной стека добавление удаление и просмотр. Реализация использует один указатель на верхний элемент, что обеспечивает постоянную скорость операций push и pop. Стек особенно полезен для задач связанных с отменой действий, где последняя операция отменяется первой, для отслеживания вызовов функций в программировании и для проверки корректности скобок в выражениях, где важно соблюдение вложенности.

Результаты работы программы

Результаты работы программы показаны на рисунках 1-12.

I. 3LR_1T.cpp (1 задание)

```
Реализация приоритетной очереди

Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 1
Введите название объекта: Лето
Введите приоритет: 1
```

Рисунок 1 – Добавление элемента в очередь

```
Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 2
Содержимое приоритетной очереди:
Имя: Зима, Приоритет: 2
Имя: Лето, Приоритет: 1
```

Рисунок 2 – Содержимое очереди

```
Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 3
Удален элемент: Зима (приоритет: 2)
```

Рисунок 3 – Удаление элемента с наивысшим приоритетом

```

Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 2
Содержимое приоритетной очереди:
Имя: Лето, Приоритет: 1

```

Рисунок 4 – Содержимое очереди после удаления элемента

```

Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 5
Неверный выбор. Введите число от 1 до 4.

```

Рисунок 5 – Ввод некорректного числа при выборе действия

```

Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 1
Введите название объекта: Осень
Введите приоритет: -3
Ошибка: приоритет должен быть положительным числом.
Введите приоритет:

```

Рисунок 6 – Ввод некорректного значения приоритета при добавлении элемента

```

Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 1
Введите название объекта: Весна
Введите приоритет: 1
Ошибка: приоритет 1 уже существует. Введите уникальный приоритет.
Введите приоритет:

```

Рисунок 7 – Ввод элемента с уже существующим приоритетом

```

Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 1
Введите название объекта: Весна
Введите приоритет: 1
Ошибка: приоритет 1 уже существует. Введите уникальный приоритет.
Введите приоритет: п
Ошибка: введите целое число
Введите приоритет:

```

Рисунок 8 – Ввод буквы в поле приоритета

```

Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 2
Список пуст

```

Рисунок 9 – Пустой список

```

Меню:
1. Добавить элемент
2. Просмотреть очередь
3. Удалить элемент с высшим приоритетом
4. Выход
Выберите действие: 4
Выход...

```

Рисунок 10 – Корректное завершение работы программы

II. 3LR_2T.cpp (2 задание)

```

Реализация очереди (FIFO)

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 1
Введите название объекта: Марина
Элемент добавлен в очередь: Марина

```

Рисунок 11 – Добавление элемента в очередь

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 3
Первый элемент: Марина

```

Рисунок 12 – Вывод первого элемента из очереди


```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 4
Содержимое очереди:
Марина
Олеся
Екатерина

```

Рисунок 13 – Вывод всех элементов в очереди

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 2
Удален элемент: Марина

```

Рисунок 14 – Удаление элемента из очереди

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 3
Первый элемент: Олеся

```

Рисунок 15 – Вывод первого элемента из очереди после удаления предыдущего

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 4
Содержимое очереди:
Олеся
Екатерина

```

Рисунок 16 – Вывод всех элементов из очереди после удаления предыдущего

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 6
Неверный выбор. Введите число от 1 до 5.

```

Рисунок 17 – Ввод некорректного числа при выборе действия

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 4
Очередь пуста

```

Рисунок 18 – Пустая очередь

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть первый элемент
4. Просмотреть всю очередь
5. Выход
Выберите действие: 5
Выход...

```

Рисунок 19 – Корректное завершение работы программы

III. 3LR_3T.cpp (3 задание)

```

Реализация стека (LIFO)
Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть верхний элемент
4. Просмотреть весь стек
5. Проверить пустоту стека
6. Выход
Выберите действие: 1
Введите название объекта: Красный
Элемент добавлен в стек: Красный

```

Рисунок 20 – Добавление элемента в стек

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть верхний элемент
4. Просмотреть весь стек
5. Проверить пустоту стека
6. Выход
Выберите действие: 4
Содержимое стека (сверху вниз):
Зеленый
Синий
Красный

```

Рисунок 21 – Вывод всех элементов стека

```
Меню:  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть верхний элемент  
4. Просмотреть весь стек  
5. Проверить пустоту стека  
6. Выход  
Выберите действие: 3  
Верхний элемент: Зеленый
```

Рисунок 22 – Вывод верхнего элемента стека

```
Меню:  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть верхний элемент  
4. Просмотреть весь стек  
5. Проверить пустоту стека  
6. Выход  
Выберите действие: 2  
Удален элемент: Зеленый
```

Рисунок 23 – Удаление верхнего элемента

```
Меню:  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть верхний элемент  
4. Просмотреть весь стек  
5. Проверить пустоту стека  
6. Выход  
Выберите действие: 7  
Неверный выбор. Введите число от 1 до 6.
```

Рисунок 24 – Ввод некорректного числа при выборе действия

```
Меню:  
1. Добавить элемент  
2. Удалить элемент  
3. Просмотреть верхний элемент  
4. Просмотреть весь стек  
5. Проверить пустоту стека  
6. Выход  
Выберите действие: 5  
Стек не пуст
```

Рисунок 25 – Проверка заполненности стека (заполнен)

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть верхний элемент
4. Просмотреть весь стек
5. Проверить пустоту стека
6. Выход
Выберите действие: 5
Стек пуст

```

Рисунок 26 – Проверка заполненности стека (пуст)

```

Меню:
1. Добавить элемент
2. Удалить элемент
3. Просмотреть верхний элемент
4. Просмотреть весь стек
5. Проверить пустоту стека
6. Выход
Выберите действие: 6
Выход...

```

Рисунок 27 – Корректное завершение работы программы

Выводы

Все три программы демонстрируют практическую реализацию основных динамических структур данных на языке С с использованием связанных списков. Каждая программа успешно решает свою задачу: приоритетная очередь обеспечивает упорядоченное хранение элементов по приоритету, обычная очередь реализует принцип FIFO (первым пришел - первым вышел), а стек работает по принципу LIFO (последним пришел - первым вышел). Общими для всех программ являются механизмы обработки ввода данных с валидацией и защитой от ошибок, включая проверку уникальности вводимых значений, контроль за выделением и освобождением динамической памяти.